

Chapter 18

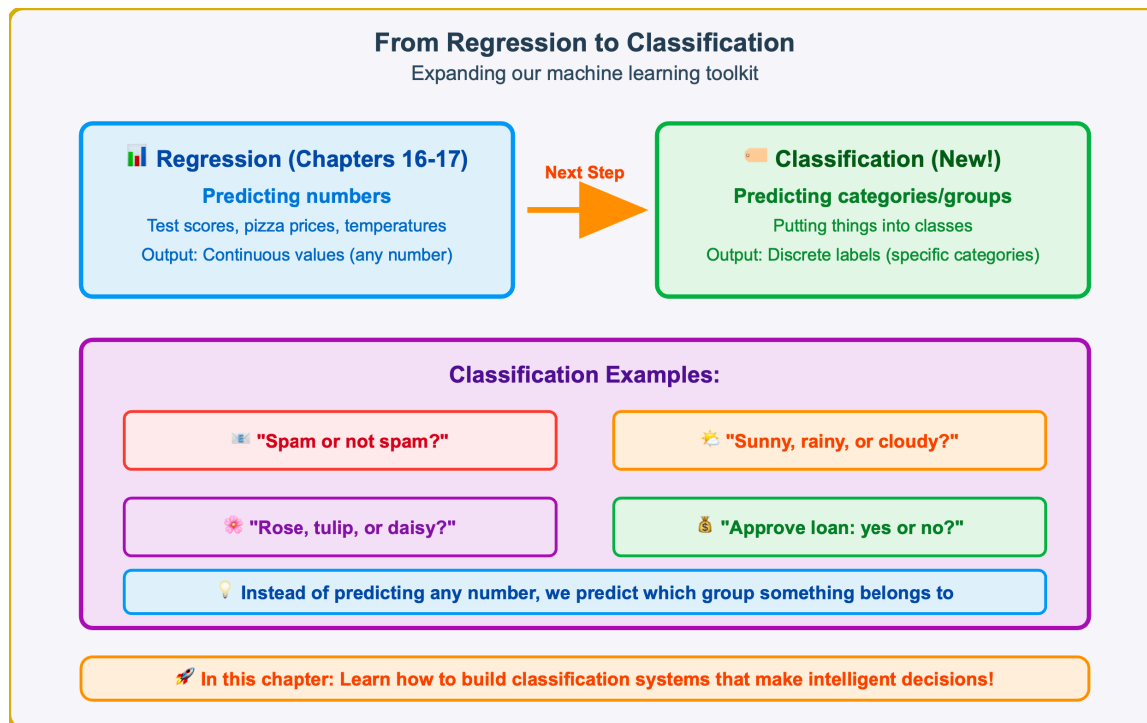


Figure 18.1 — From Regression to Classification — Machine Learning Progression. This introduction shows the natural progression from regression (predicting numbers) to classification (predicting categories), illustrating how machine learning expands from continuous value prediction to discrete category assignment with practical examples.

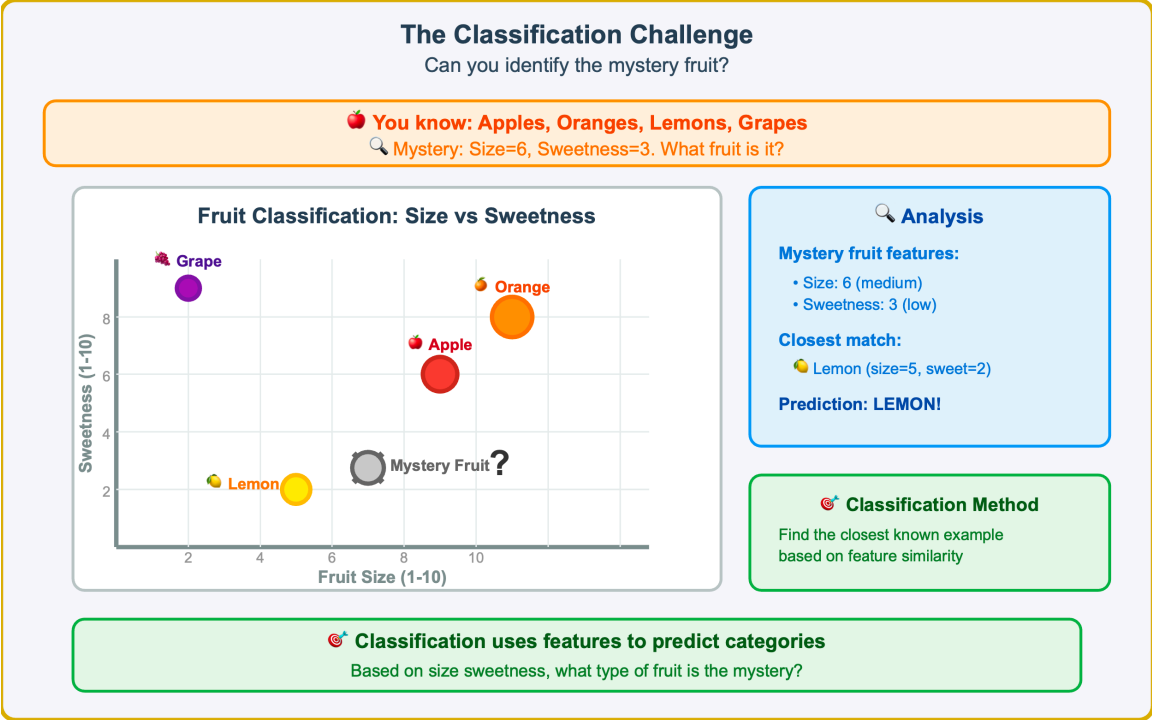


Figure 18.2 — The Classification Challenge — Feature—Based Prediction. This interactive example demonstrates the fundamental concept of classification by asking students to identify a mystery fruit based on its size and sweetness features, showing how algorithms use similarity to known examples for category prediction.

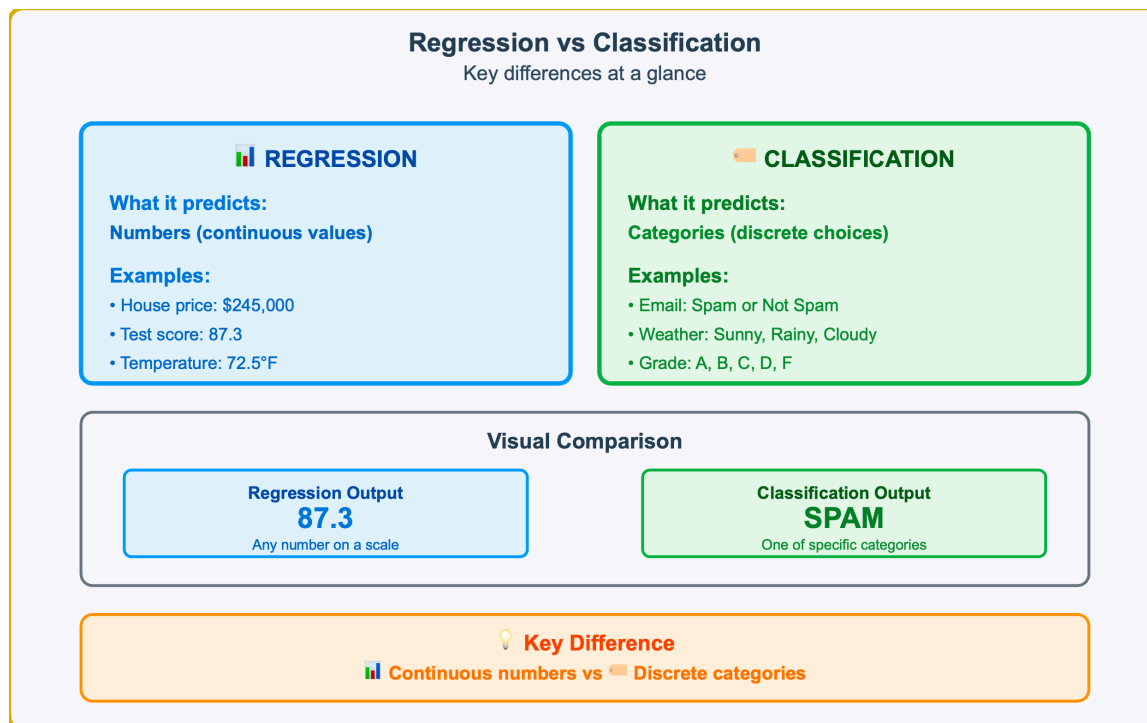


Figure 18.3 — Regression vs Classification — Core Concept Comparison. This side—by—side comparison clearly illustrates the fundamental difference between regression (predicting continuous numerical values) and classification (predicting discrete categories), with concrete examples and visual output representations.

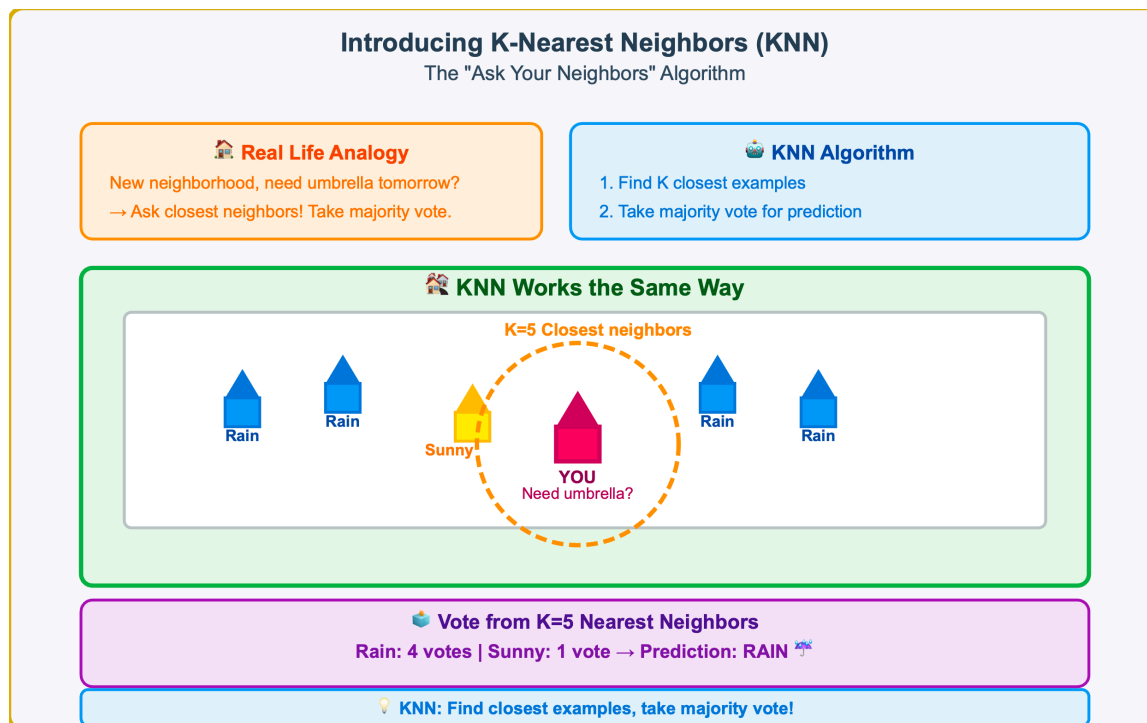


Figure 18.4 — Introducing K—Nearest Neighbors (KNN) — The Ask Your Neighbors Algorithm. This intuitive analogy demonstrates how KNN classification works by comparing it to asking neighbors for advice: find the K closest examples to your new data point and take a majority vote to make the prediction.

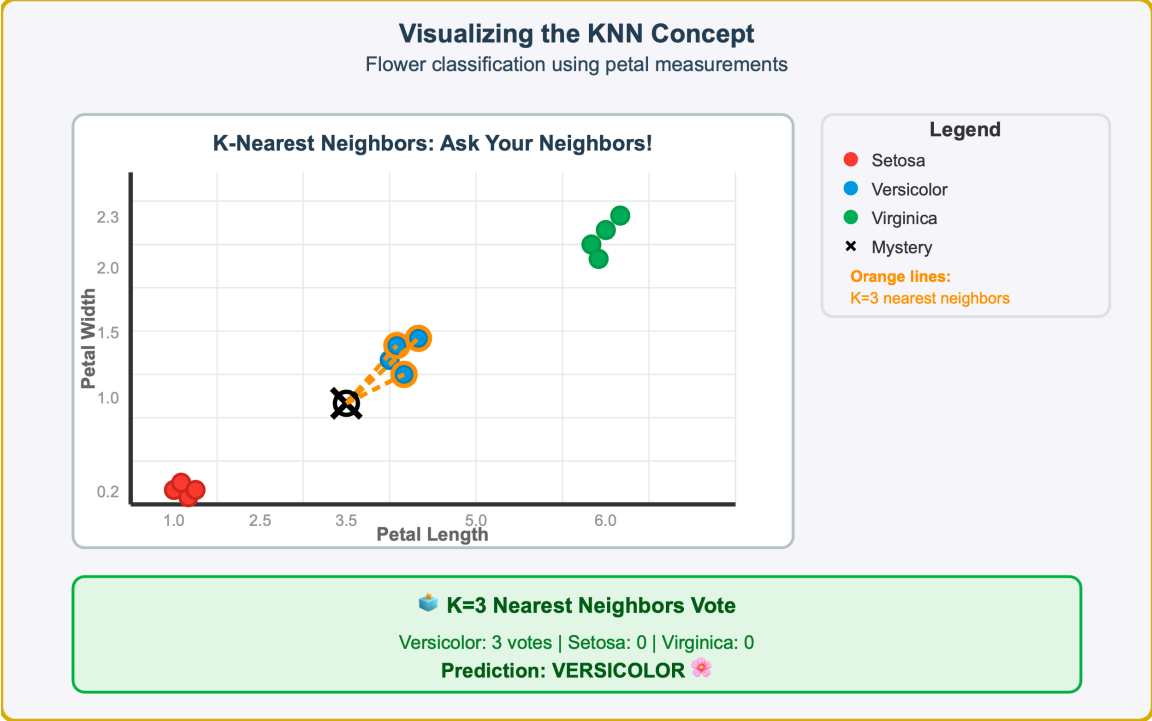


Figure 18.5 — Visualizing the KNN Concept — Flower Classification Example. This scatter plot demonstrates how KNN classification works by plotting flowers according to their petal measurements. The algorithm finds the K=3 nearest neighbors to the mystery flower (shown with orange dashed lines) and predicts its species based on majority vote.

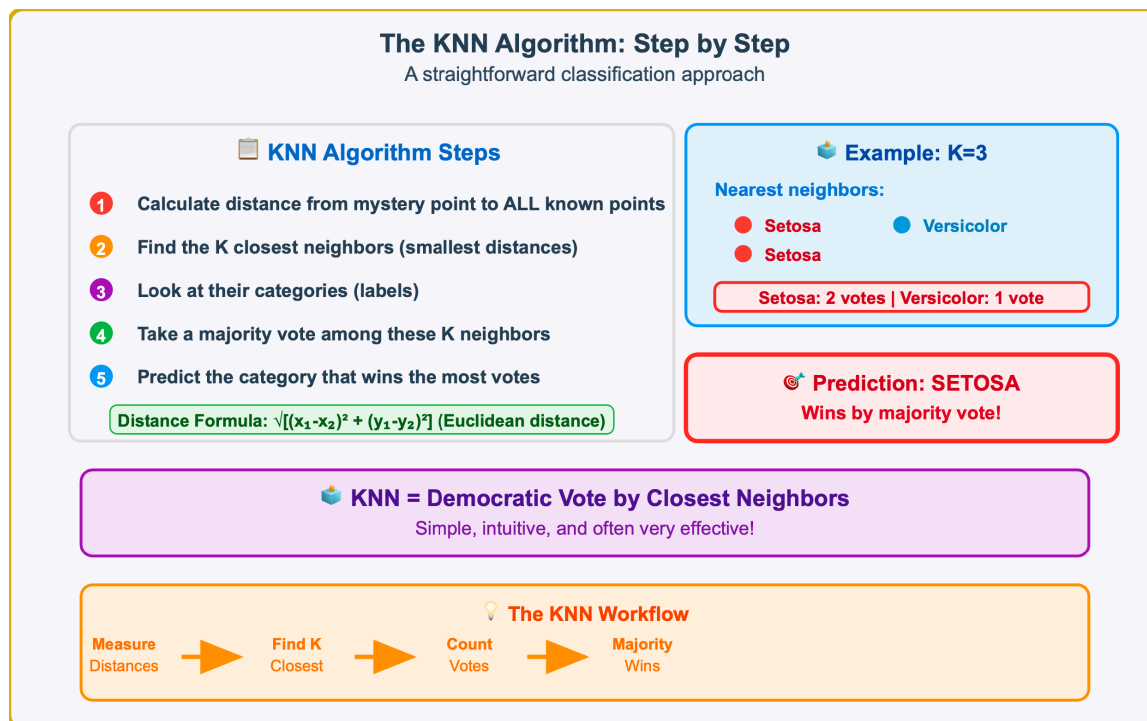


Figure 18.6 — The KNN Algorithm — Step by Step Implementation Guide. This comprehensive breakdown shows the five steps of KNN classification with a concrete K=3 example, demonstrating how the algorithm measures distances, identifies neighbors, counts votes, and makes predictions through democratic decision-making.

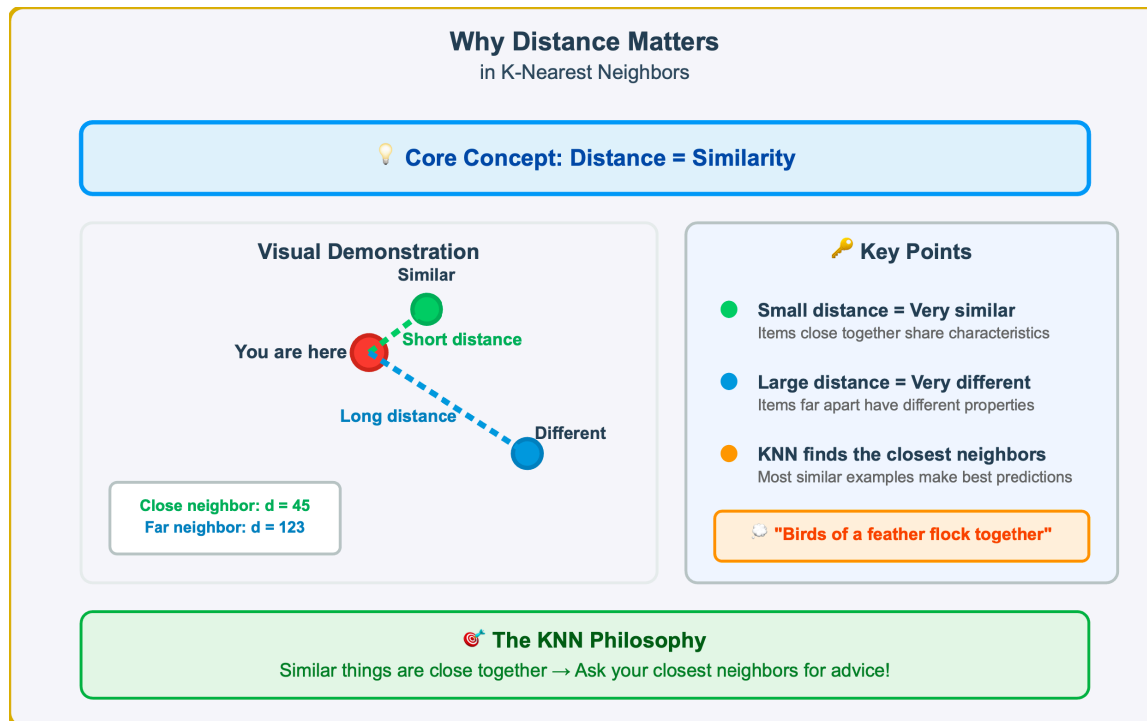


Figure 18.7 — Why Distance Matters in K—Nearest Neighbors — Similarity Through Proximity. This conceptual diagram illustrates the fundamental principle that distance represents similarity in KNN: closer points are more similar and therefore better predictors, while distant points are less relevant for classification decisions.

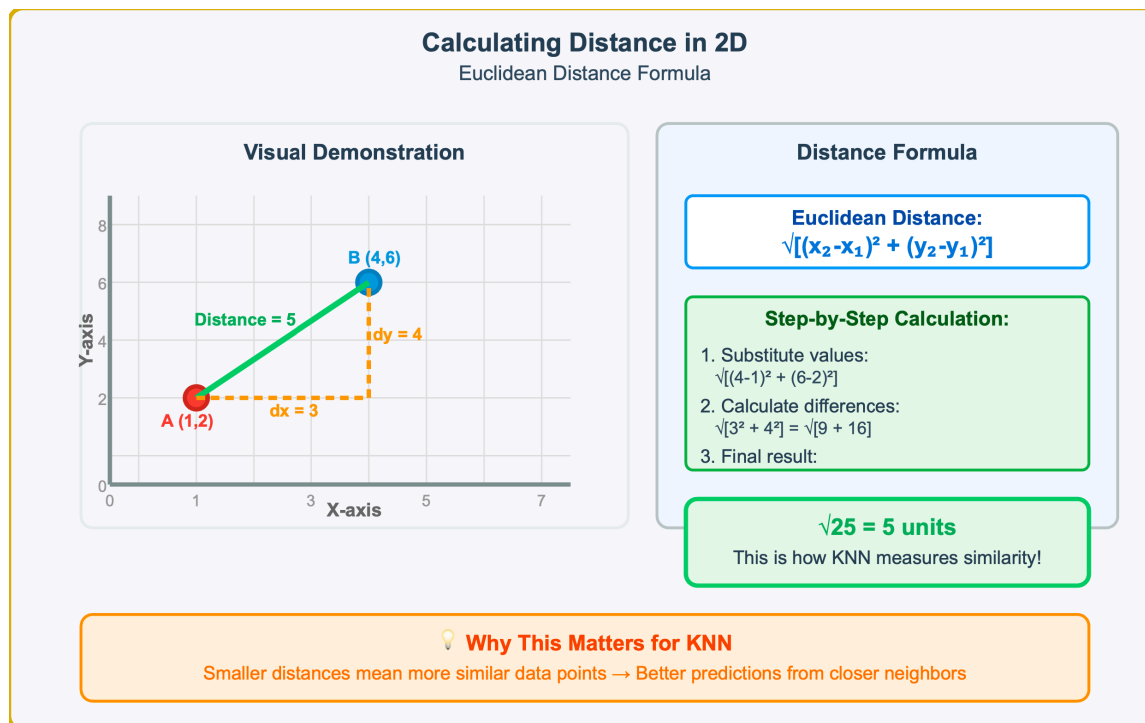


Figure 18.8 — Calculating Distance in 2D — Euclidean Distance Formula. This step—by—step demonstration shows how to calculate the Euclidean distance between two points using the Pythagorean theorem, illustrating the mathematical foundation that KNN uses to measure similarity between data points.

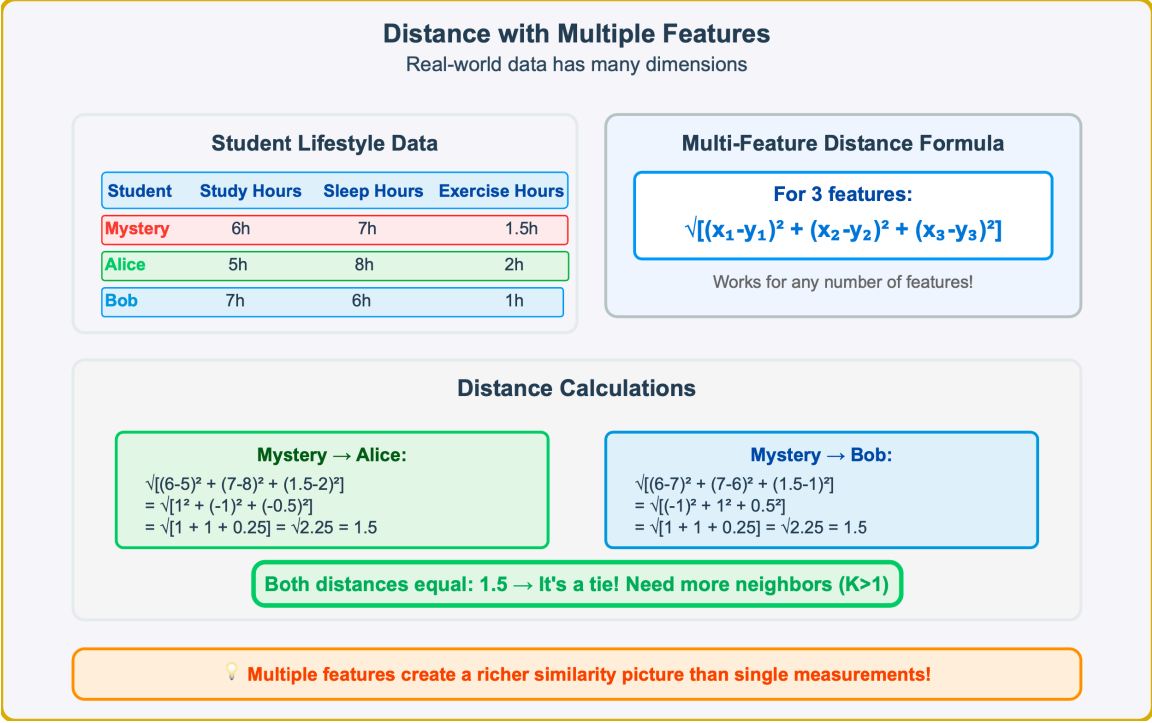


Figure 18.9 — Distance with Multiple Features — Multi—Dimensional Similarity. This example demonstrates how KNN calculates distance when data points have multiple characteristics (study hours, sleep hours, exercise hours), showing how the Euclidean distance formula extends to handle any number of features for more comprehensive similarity measurement.

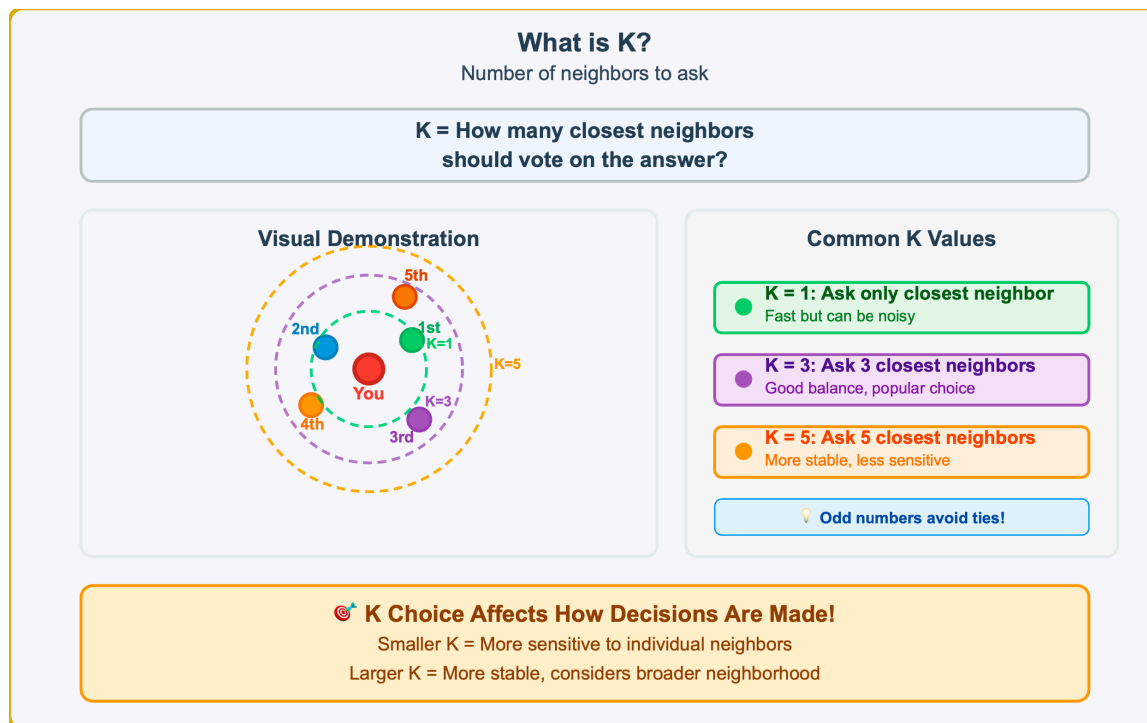


Figure 18.10 — What is K? — Understanding the Neighborhood Size Parameter. This visualization explains the crucial K parameter in KNN by showing how different values (K=1, K=3, K=5) determine how many nearest neighbors participate in the voting process, illustrating the trade—off between sensitivity and stability in classification decisions.

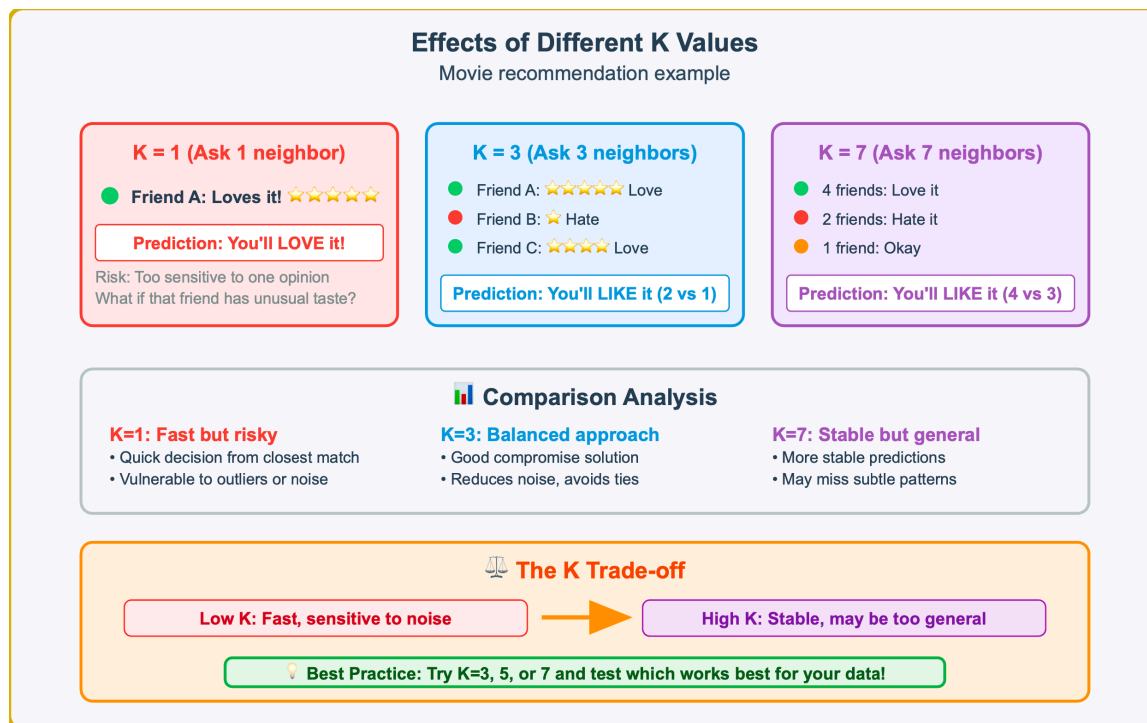


Figure 18.11 — Effects of Different K Values — Movie Recommendation Trade—offs. This practical example demonstrates how different K values affect KNN predictions using a relatable movie recommendation scenario, illustrating the fundamental trade—off between sensitivity (low K) and stability (high K) in classification decisions.

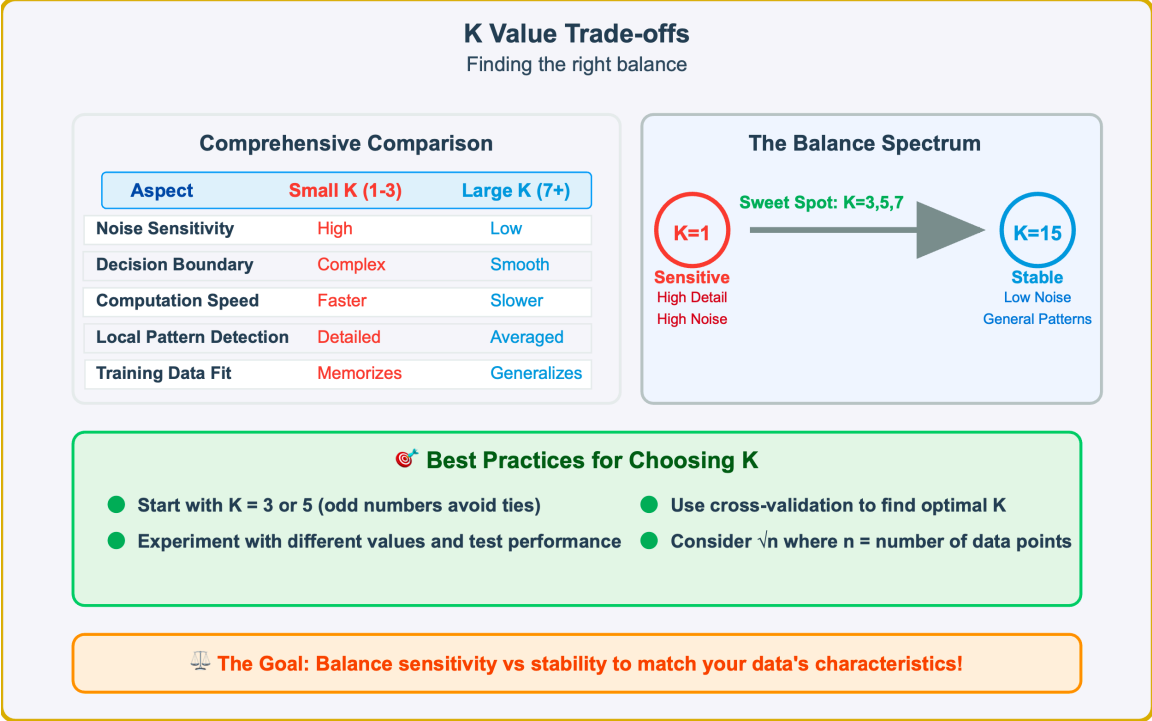


Figure 18.12 — K Value Trade—offs — Balancing Sensitivity and Stability. This comprehensive comparison illustrates the fundamental trade—offs between small K values (sensitive, detailed, noisy) and large K values (stable, smooth, general), along with practical guidelines for selecting appropriate K values in real applications.

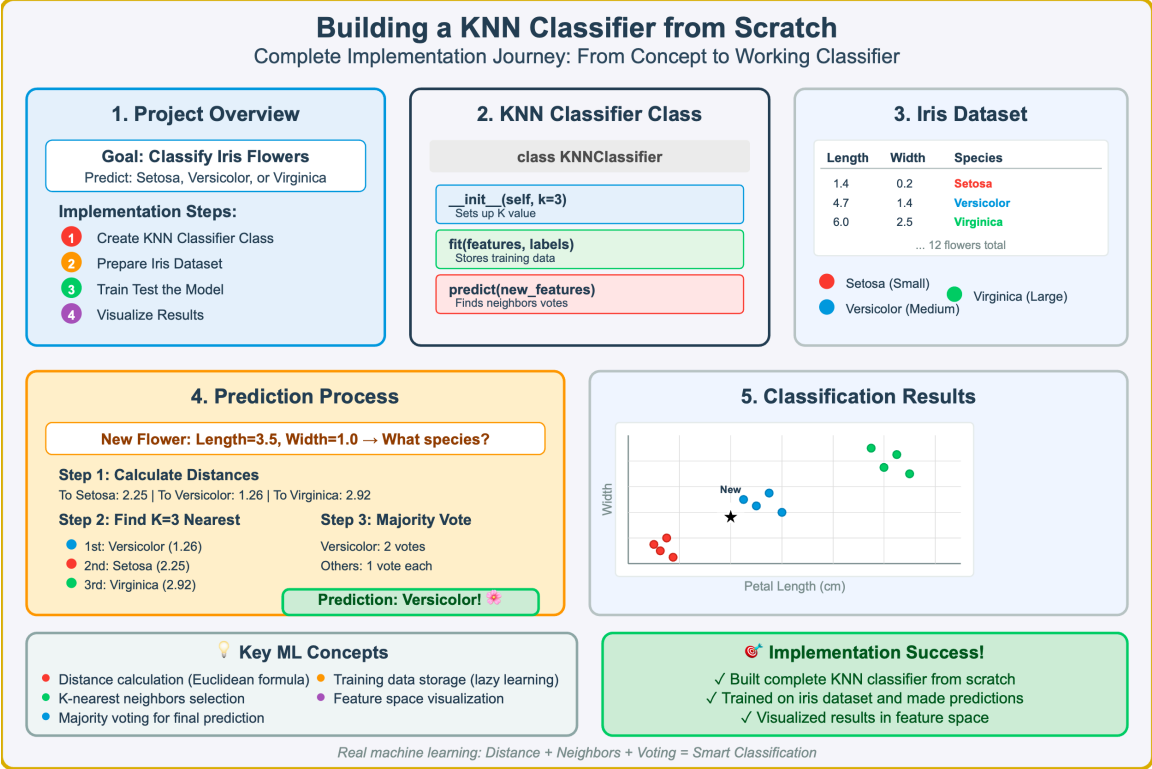


Figure 18.13 — Building a KNN Classifier from Scratch — Complete Implementation Journey. This comprehensive walkthrough shows the five essential stages of creating a machine learning classifier: (1) Project overview and goals, (2) Class structure design, (3) Dataset preparation with iris flowers, (4) Step—by—step prediction process using distance calculations and majority voting, and (5) Visual results in feature space. The implementation demonstrates core ML concepts including lazy learning, feature space representation, and k—nearest neighbor decision making.

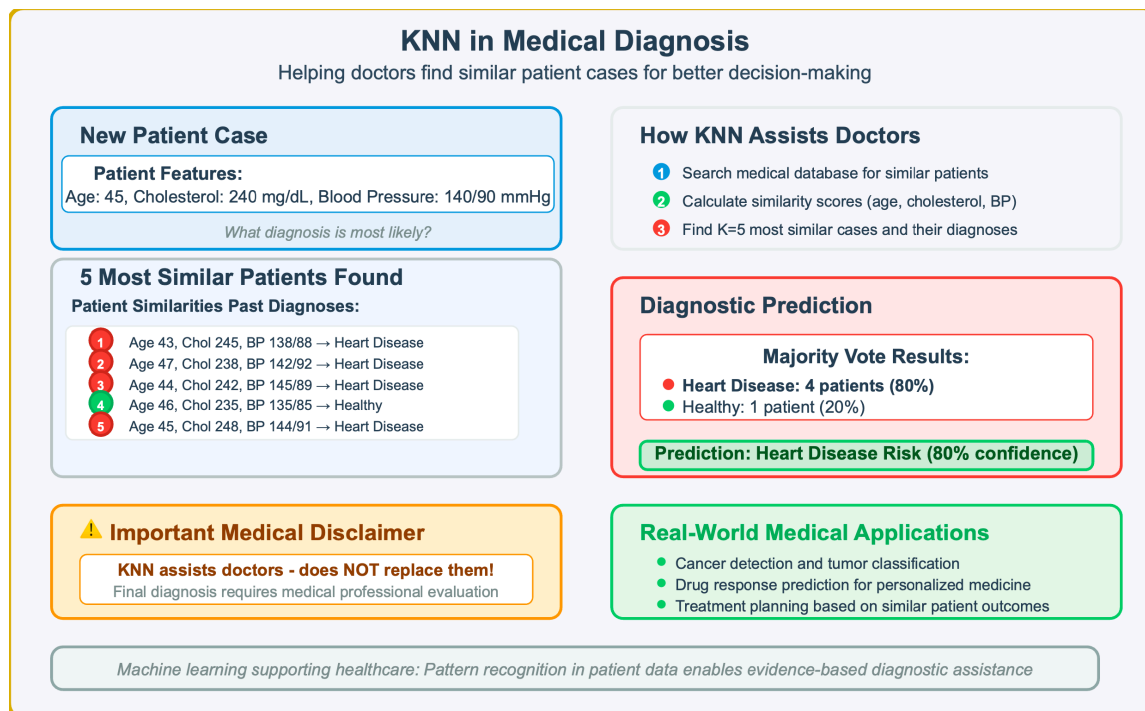


Figure 18.14 — KNN in Medical Diagnosis — Assisting Healthcare Decisions. This example demonstrates how K—nearest neighbor algorithms can assist doctors by finding similar patient cases from medical databases, calculating similarities based on patient features, and providing diagnostic predictions with confidence levels. The system supports but does not replace medical expertise.

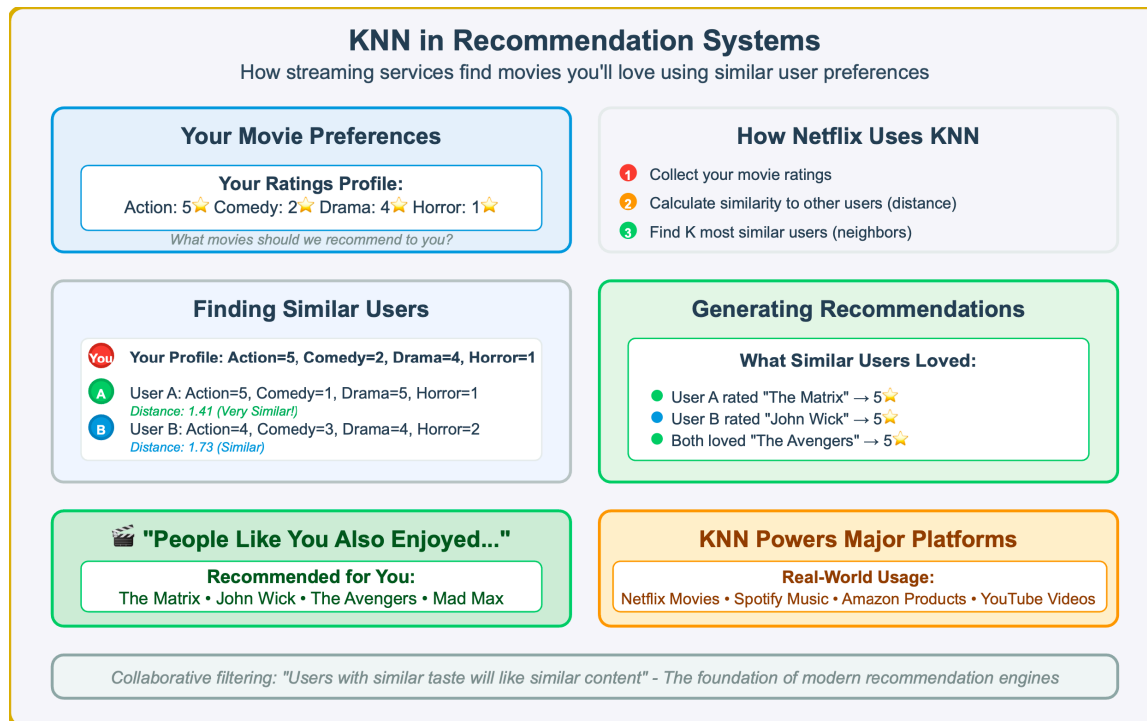


Figure 18.15 — KNN in Recommendation Systems — How Netflix Finds Movies You’ll Love. This example demonstrates how streaming services use K—nearest neighbor algorithms to find users with similar viewing preferences, calculate taste similarity using rating distances, and recommend content based on what similar users enjoyed. The system powers personalized recommendations across major platforms.

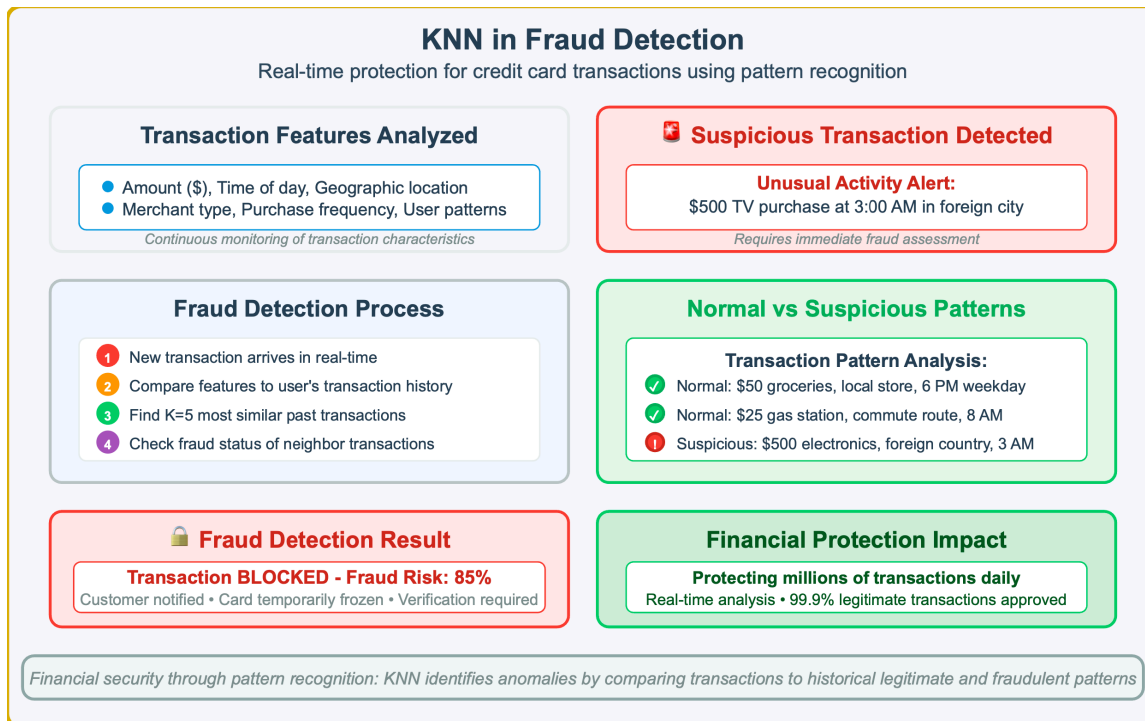


Figure 18.16 — KNN in Fraud Detection — Protecting Credit Card Transactions. This example demonstrates how financial institutions use K—nearest neighbor algorithms to analyze transaction patterns, compare new transactions against historical data using features like amount, time, and location, and identify potentially fraudulent activity by finding similar past transactions and checking their fraud status.

Try It Yourself!

Hands-On Challenge: Student Performance Predictor

Your Mission

Build a Grade Predictor using KNN

Predict student grades (A, B, C, D) based on:
Study hours • Sleep hours • Social media hours

Training Data Sample

Study	Sleep	Social	Grade	+ more data...
8h	8h	1h	A	2h, 4h, 7h → D
6h	9h	1h	B	5h, 6h, 2h → C

New Student to Predict

Mystery Student Profile:

Study: 6 hours • Sleep: 7 hours • Social Media: 3 hours
What grade will they get? 🤔

Your Challenge Steps

- Create KNNClassifier(k=3)
- Train with .fit(features, grades)
- Predict new student's grade
- Analyze your results

Code Template

```
# Your code here:
classifier = KNNClassifier(k=3)
classifier.fit(features, grades)
prediction = classifier.predict([6,7,3])
```

Bonus Challenges

- Try different K values (K=1, K=5, K=7)
- Create a scatter plot of your results
- Test multiple student profiles
- Analyze which features matter most

🚀 Ready to put your KNN skills to the test? You've got this! 📚✨

Figure 18.17 — Hands—On Challenge — Student Performance Predictor. This interactive exercise challenges students to apply their KNN knowledge by building a grade prediction system using study hours, sleep hours, and social media usage as features. Students create a KNN classifier, train it on sample data, and predict grades for new students while experimenting with different K values.



Advanced Challenge!

Weather Prediction System - Level Up Your KNN Skills



Your Weather Mission

Build a Weather Forecasting System

Predict weather conditions using atmospheric data:

☀️ Sunny • 🌧️ Rainy • ⚡️ Stormy



Atmospheric Features

🌡️ **Temperature**

Range: 30°F - 78°F

💧 **Humidity**

Range: 55% - 95%

🌀 **Wind Speed**

Range: 3 - 30 mph



Weather Pattern Data

Training Pattern Examples:

- ☀️ Sunny: Warm (70-80°F), Low humidity (55-65%), Light wind (3-8 mph)
- 🌧️ Rainy: Cool (40-50°F), High humidity (80-90%), Moderate wind (8-15 mph)
- ⚡️ Stormy: Variable temp, Very high humidity (90-95%), Strong wind (15-30 mph)



Mystery Weather Conditions

Current Atmospheric Readings:

🌡️ Temperature: 55°F

💧 Humidity: 70%

🌀 Wind: 10 mph

What's the weather forecast? 🤔



Challenge Levels

1 **Basic:** Build classifier and predict weather condition

2 **Advanced:** Add confidence levels and probability scores



Implementation Tips

- Normalize features (temperature, humidity, wind speed)
- Experiment with K=3, K=5, K=7 for best accuracy

🚀 Ready to become a weather forecasting expert? Build your meteorological ML system! ☁️

Figure 18.18 — Advanced Challenge — Weather Prediction System. This extended challenge builds on KNN fundamentals by tasking students with creating a weather classification system. Students use temperature, humidity, and wind speed as features to predict weather conditions (sunny, rainy, or stormy), with options for basic implementation or advanced features like confidence levels.

Interactive Gaming Challenge!

Animal Classification Game - Make Learning Fun with KNN

Your Gaming Mission

Create an Interactive Animal Guessing Game

Players input animal features → KNN predicts the animal
Fun + Educational = Perfect Learning! 🎮

Animal Features (1-10 Scale)

Size: 1 = tiny (mouse) → 10 = huge (whale)

Speed: 1 = slow (sloth) → 10 = super fast (cheetah)

Diet: 1 = herbivore (grass) → 5 = carnivore (meat)

Animal Database Examples

Training Animals:

- Rabbit: Size=2, Speed=8, Diet=1 (small, fast herbivore)
- Lion: Size=8, Speed=7, Diet=5 (large, fast carnivore)
- Elephant: Size=10, Speed=3, Diet=1 (huge, slow herbivore)

Game Code Template

```
# Interactive game loop:
features = input("Size, Speed, Diet: ")
```

Interactive Game Flow

How the Game Works:

- 1 Player inputs: "Enter size (1-10):", "Enter speed:", "Enter diet:"
- 2 KNN analyzes and predicts: "I think it's a Lion! 🦁"
- 3 Show reasoning: "Based on similarity to other large carnivores"

Advanced Game Features

- Let players add new animals to the database
- Show confidence scores and top 3 guesses

Ready to gamify machine learning? Create the ultimate animal guessing game! 🦁🐘🐰

Figure 18.19 — Interactive Challenge — Animal Classification Game. This gamified learning experience challenges students to create an interactive animal guessing game using KNN classification. Users input animal features (size, speed, diet) on a 1—10 scale, and the system predicts the animal while explaining its reasoning based on similar animals in the database.

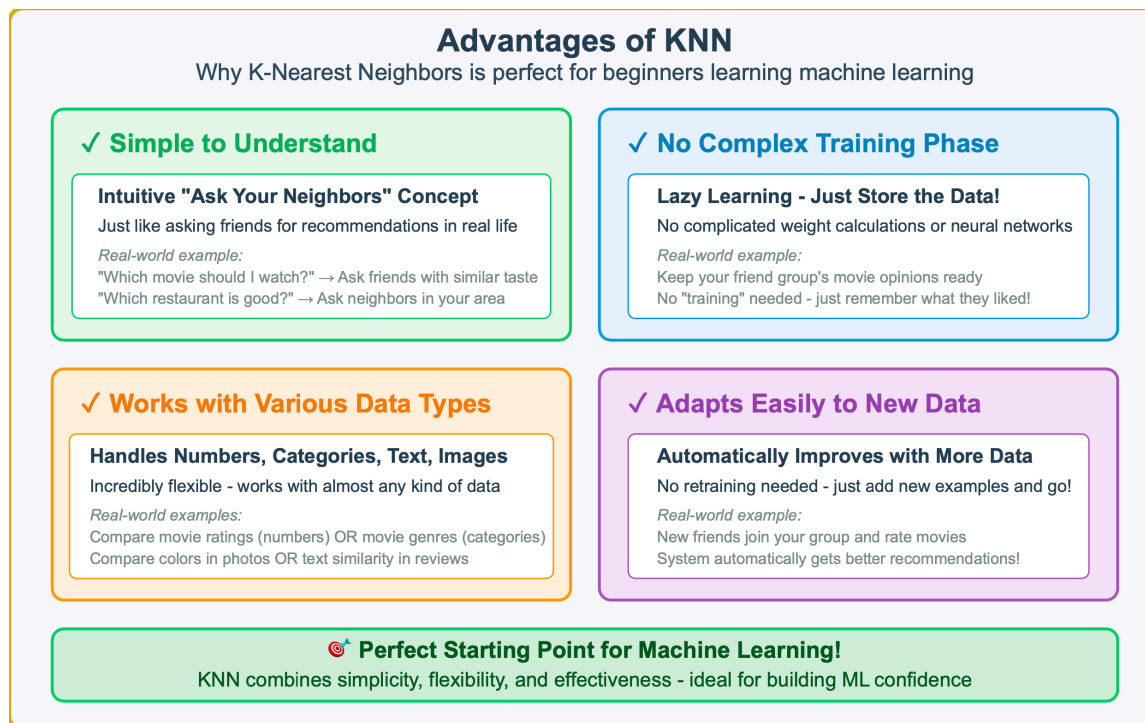


Figure 18.20 — Advantages of KNN — Why It’s Perfect for Beginners. This summary highlights the key benefits that make K—nearest neighbors an ideal starting algorithm for machine learning education: intuitive “ask your neighbors” concept, no complex training phase, compatibility with various data types, and automatic adaptation to new data.

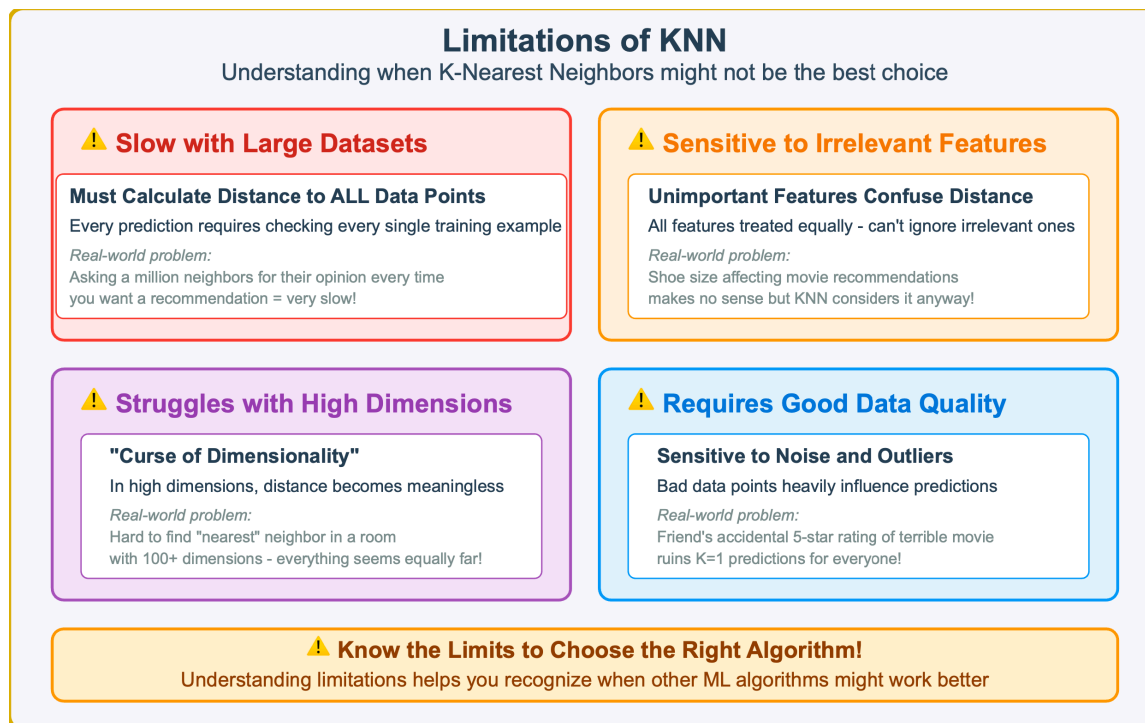


Figure 18.21 — Limitations of KNN — When to Consider Alternative Algorithms. This analysis identifies key constraints of K—nearest neighbors: computational slowness with large datasets, sensitivity to irrelevant features, struggles with high—dimensional data (curse of dimensionality), and vulnerability to data quality issues like noise and outliers.